

Pruebas de Software: Integración CI/CD (Informe Presentación 2)

Integrantes: Martin Abbott

Carlos Jara

Esteban Manríquez

Link Video Presentación:

<https://youtu.be/bxFnpli9R0Q>

Link Github:

<https://github.com/windu2/medfinder-ci-cd>

Link Jira:

<https://jenkins-medfinder.atlassian.net/jira/software/projects/JMF/boards/1>

1.- Introducción

A partir de lo realizado en la entrega anterior, que comprendió la realización de un directorio médico con sus principales funcionalidades (CRUD), se debe implementar un entorno *CI/CD* para la realización de pruebas de *FrontEnd* mediante el uso de *Selenium*, que es una herramienta muy utilizada para llevar a cabo el testeo de Plataformas Web.

Para lograr implementar *CI/CD* se debe utilizar Jenkins como herramienta para definir la Pipeline del proyecto, también se debe usar *gitflow* para tener control de versiones y dividir el trabajo del proyecto en la plataforma de Jira. Todo lo anterior debe estar conectado a Slack, el cuál servirá de herramienta para mantener al equipo informado acerca del trabajo realizado.

La idea de este proyecto es aprender de los beneficios que implica trabajar con un entorno *CI/CD* y preparar a los integrantes del equipo para futuros proyectos en donde se utilice esta forma de desarrollo orientada a la entrega y despliegue continuo de funcionalidades.

2.- Supuestos

- Al llenar el campo *credencial*, se asume que se colocará un link a un documento PDF existente.
- Al llenar el campo *ubicación*, debe hacerse mediante un link de google maps (Específicamente el link que se obtiene mediante la funcionalidad compartir ubicación de google maps).
- A la hora de ingresar una *especialidad*, se asume que se escribirá alguna de las existentes dentro del sistema.
- Solamente un administrador logueado dentro del sistema podrá agregar, cambiar o eliminar información de los médicos de la plataforma.

3.- Trabajo Realizado

3.1 Proyecto

En un principio, se hizo la conexión entre las distintas herramientas requeridas para llevar a cabo la pipeline y comunicación del equipo. En este caso, se estableció la conexión de notificaciones a Slack por parte de Jenkins, Github y Jira. En el caso de la Pipeline, se estableció el enlace entre Github y Jenkins para que actualizara y ejecutara las pruebas cuando se realizara un *push* al repositorio.

Una vez configurada la mayor parte de las herramientas, se procedió a investigar acerca de la versión de Selenium que sería prudente usar, ya que actualmente existen distintas versiones para distintos entornos y lenguajes de programación. Bajo el contexto de que la mayoría de los integrantes tenían instalado python en su Sistema Operativo, se estableció la versión Python de Selenium como la alternativa definitiva para ocupar.

Una vez escogida la versión de la herramienta se procedió a realizar la escritura de las pruebas en *Selenium*, las cuales están basadas en la entrega previa. Cabe recalcar que para el desarrollo de las pruebas se utilizó GitFlow para realizar la distribución del trabajo y mantener una gestión de las versiones desarrolladas.

Finalmente, se realizaron las pruebas en conjunto dentro de la rama de “*Dev*”, donde se realizaron los últimos “*hotfixes*” para luego dejar la entrega definitiva dentro de la rama del “*Main*”.

3.2 Pruebas

Para realizar las pruebas, se ocupó lo definido en la presentación anterior. Las pruebas identificadas fueron las siguientes:

Pruebas del sistema CRUD

P1: Prueba unitaria de creación de registro:

- 1.1 Se crea un registro ficticio y se llama al método de creación correspondiente.
- 1.2 Se verifica que el registro se haya guardado correctamente en la base de datos.
- 1.3 Se verifica que los datos del registro creado sean los mismos que los proporcionados en la entrada.

P2: Prueba unitaria de lectura de registro:

- 2.1 Se agrega un registro ficticio a la base de datos.
- 2.2 Se llama al método de lectura correspondiente y se proporciona el ID del registro creado anteriormente.

2.3 Se verifica que el registro devuelto tenga los mismos valores que el registro creado anteriormente.

P3: Prueba unitaria de actualización de registro:

- 3.1 Se crea un registro ficticio y se guarda en la base de datos.
- 3.2 Se modifica algún campo del registro creado anteriormente y se llama al método de actualización correspondiente.
- 3.3 Se verifica que los cambios se hayan guardado correctamente en la base de datos.
- 3.4 Se verifica que los datos del registro actualizado sean los mismos que los proporcionados en la entrada.

P4: Prueba unitaria de eliminación de registro:

- 4.1 Se crea un registro ficticio y se guarda en la base de datos.
- 4.2 Se llama al método de eliminación correspondiente y se proporciona el ID del registro creado anteriormente.
- 4.3 Se verifica que el registro se haya eliminado correctamente de la base de datos.
- 4.4 Se verifica que el registro eliminado ya no esté presente en la base de datos al intentar leerlo.

Pruebas de Valores Límite/ Supuestos

P5: Validar Rut (Según fórmula)

- 5.1 Se ingresa un registro ficticio de un médico con un rut inválido (Según fórmula).
- 5.2 Se verifica que no se haya realizado el post del registro al Back-End.
- 5.3 Se ingresa un registro ficticio de un médico con un rut válido (Según fórmula).
- 5.4 Se verifica que se haya realizado el post del registro al Back-End.
- 5.5 Se verifica que los datos del registro creado sean los mismos que los proporcionados en la entrada.

P6: Validar Rut (Existe o no Dentro de la Base de Datos)

- 6.1 Se ingresa un registro ficticio de un médico con un rut que ya exista dentro de la Base de Datos.
- 6.2 Se verifica que el sistema entregue feedback de que el rut proporcionado está actualmente en uso.
- 6.3 Se ingresa un registro ficticio de un médico con un rut válido (Según fórmula), y que no exista en la Base de Datos.
- 6.4 Se verifica que se haya realizado el post del registro al Back-End.
- 6.5 Se verifica que los datos del registro creado sean los mismos que los proporcionados en la entrada.

P7: Validar Edad

- 7.1 Se ingresa un registro ficticio de un médico con una edad igual a 0.
- 7.2 Se verifica que el sistema entregue feedback de que la edad entregada no es correcta.
- 7.3 Se ingresa un registro ficticio de un médico con una edad igual a 100.
- 7.4 Se verifica que el sistema entregue feedback de que la edad entregada no es correcta.
- 7.5 Se ingresa un registro ficticio de un médico con una edad igual a 1.
- 7.6 Se verifica que se haya realizado el post del registro al Back-End.

7.7 Se verifica que los datos del registro creado sean los mismos que los proporcionados en la entrada.

P8: Validar nombre/apellido/especialidad

8.1 Se ingresa un registro ficticio de un médico con un nombre con largo mayor a 50 caracteres.

8.2 Se verifica que el sistema no permita ingresar más de 50 caracteres al campo.

8.3 Se ingresa un registro ficticio de un médico con un nombre con largo menor a 50 caracteres con un espacio al inicio.

8.4 Se verifica que el sistema entregue feedback de que el nombre entregado no es correcto.

8.5 Se ingresa un registro ficticio de un médico con un nombre con largo menor a 50 caracteres que contenga números.

8.6 Se verifica que el sistema entregue feedback de que el nombre entregado no es correcto.

8.7 Se ingresa un registro ficticio de un médico con un nombre con largo menor a 50 caracteres que contenga un tilde.

8.8 Se verifica que el sistema entregue feedback de que el nombre entregado no es correcto.

8.9 Se ingresa un registro ficticio de un médico con un nombre con largo menor a 50 caracteres que no contenga tildes ni espacios al inicio.

8.10 Se verifica que se haya realizado el post del registro al Back-End.

8.11 Se verifica que los datos del registro creado sean los mismos que los proporcionados en la entrada.

P9: Validar Ubicación/Credencial

9.1 Se ingresa un registro ficticio de un médico con ubicación inválida (No formato url)

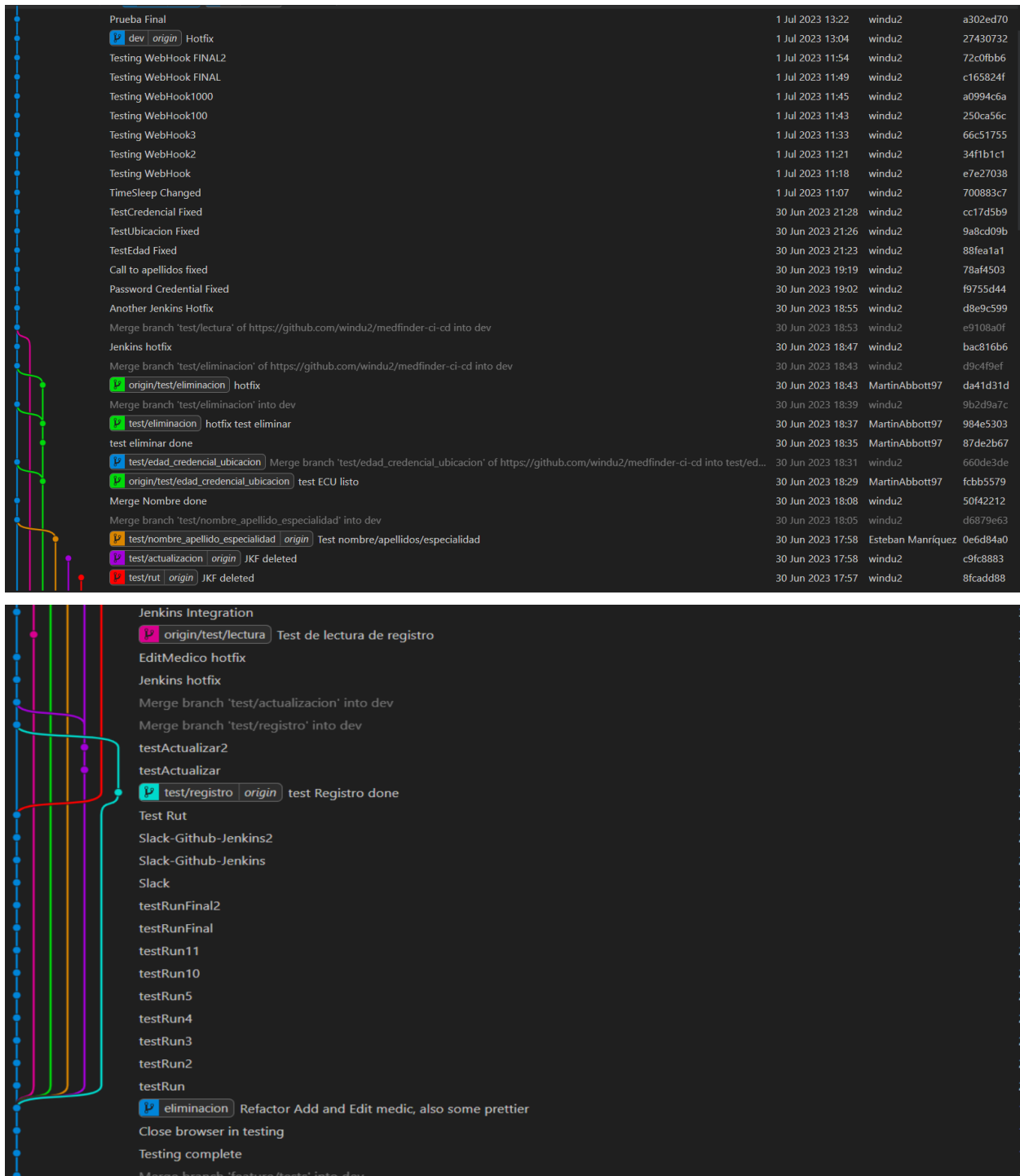
9.2 Se verifica que el sistema entregue feedback de que la ubicación entregada no es correcta.

9.3 Se ingresa un registro ficticio de un médico con ubicación válida (Formato url).

9.4 Se verifica que se haya realizado el post del registro al Back-End.

9.5 Se verifica que los datos del registro creado sean los mismos que los proporcionados en la entrada.

Anexos Gitflow



Proyectos / Jenkins-MedFinder

Cronograma

🔍  Categoría de est... ▼ Epic ▼ 🔗 Ver configuración

	JUN				JUN							JUL							JUL							
	22	23	24	25	26	27	28	29	30	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
🔍 JMF-8 Configurar Entornos																										
🔍 JMF-9 Desarrollo Pruebas																										
🔍 JMF-10 Pruebas en Conjunto																										
🔍 JMF-11 Realización de Informe																										
+ Crear Epic																										

🔍 Hoy **Semanas** Meses Trimestres ⓘ ↗

Proyectos / Jenkins-MedFinder

Kanban

Q

CJ MA EM

Epic ▾

AGRPAR POR

Nada ▾

Insights

POR HACER

+ Crear incidencia

EN CURSO

DEV

MAIN 7 INCIDENCIAS ✓

Pruebas de RUT

☒ JMF-1 ✓ CJ

Prueba de Creación de Registro

☒ JMF-4 ✓ CJ

Prueba de actualización de Registro

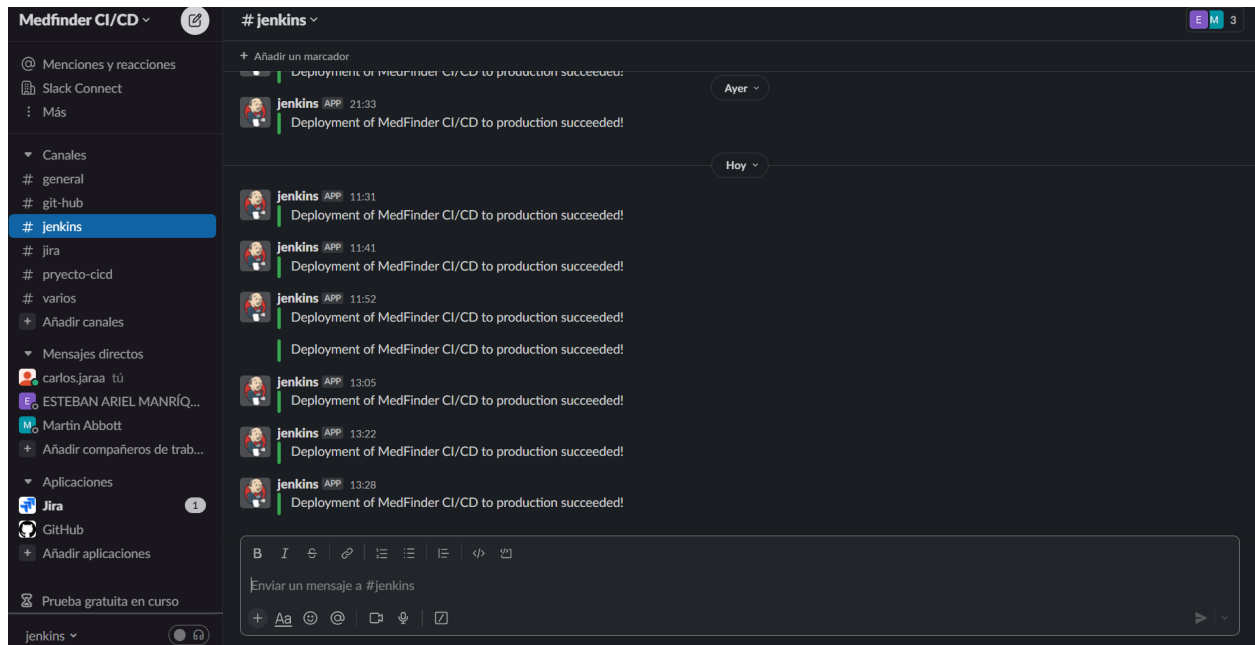
☒ JMF-6 ✓ CJ

Prueba de eliminación de registro

☒ JMF-7 ✓ MA

Pruebas de Edad / Credencial / Ubicación

Anexo Slack



(Código de Pruebas y Pipeline Anexado a .ZIP de Entrega)

4. Problemas encontrados y soluciones

La mayor problemática encontrada al momento de realizar este proyecto fue la parte de integración con Jenkins y todo lo relacionado al mismo. Esto debido a que es la primera vez que alguno de nosotros se enfrentaba a un ambiente relacionado a CI/CD por lo que finalmente la pipeline y el funcionamiento del código con Jenkins fue lo último que se terminó logrando dentro de esta entrega. En otras palabras, el enlace entre Github y Jenkins, además de la ejecución del código en el propio Jenkins.

Respecto a la parte de pruebas con Selenium, no tuvimos mayores problemas como equipo al momento de realizar las distintas pruebas. Cada integrante del equipo programó las pruebas que se le fueron asignadas previamente.

También decir que otro problema con el que nos encontramos fueron problemas de integración a la hora de realizar las pruebas, esto por los distintos navegadores y versiones de los mismos que tenía cada miembro del equipo instaladas en su equipo. Esto causó que algunas de las alertas o notificaciones fueran distintas dependiendo del navegador y su versión, generando un error de estandarización. Para resolver esto, se realizó un hotfix y se utilizó el navegador instalado en el equipo de Carlos, que fue en donde se corrieron las pruebas y grabó el video presentación.

5.- Conclusión

Los miembros del equipo fueron capaces de ejecutar las pruebas de software necesarias y correspondientes a probar las distintas funcionalidades que se integraron en MedFinder en el proyecto pasado, ingresando los valores de entrada y valores límites indicados para la correcta realización de estos, mediante la elaboración de código con python selenium y la configuración e integración apropiada de Jenkins, donde mediante el uso de su Pipeline y las configuraciones necesarias fue posible la automatización y ejecución de todas las pruebas de manera continua.

La integración de las distintas tecnologías ocupadas como Jira y Github para la orquestación, y Slack para comunicación y notificaciones fue realizada con satisfacción, y si bien el uso de Selenium Python fue relativamente fácil de aprender y usar por el grado de conocimiento que ya se tiene del lenguaje, la otra tecnología nueva que se ocupó, Jenkins, fue la que lejos nos dio más inconvenientes, ya que navegar por su interfaz para realizar cada configuración e integraciones, como por ejemplo con Github, se hizo complicado y se requirió ver una severa cantidad de tutoriales y videos para poder realizarlo, lo que trajo como consecuencia un gran consumo de tiempo. Es debido a estas complicaciones que se tuvieron al principio que el proceso no fue hecho mediante una práctica CI/CD desde el inicio, por lo que si bien al final del proyecto se logró la automatización de las pruebas realizadas, no fue hecho de la manera más idónea.

Finalmente cabe destacar que hubo una muy buena comunicación entre los integrantes del equipo, donde el medio principal de comunicación fue Discord en vez de Slack, y el trabajo realizado fue bastante eficiente en cuanto a la relación del progreso al tiempo utilizado.